Adrian Rodrigues

Senior Graphics / Rendering Programmer

adrianrodriguesm.dev@gmail.com · +351 963 643 666 · Spain / Portugal (EU) · Open to fully remote
adrianrodriguesm.com · linkedin.com/in/adrian-rodrigues-437b0a115 · github.com/adrianrodriguesm

SUMMARY

Senior Graphics Programmer with 4+ years in AAA game development on Unreal Engine 4 and 5, shipping to PC, PS5 and Xbox Series X|S. Specialized in rendering performance, memory and stability: GPU frame-time optimization of Nanite, Virtual Shadow Maps and Virtual Textures, memory optimization within fixed console budgets, and engine-level fixes down to the platform-specific RHI. Builds Cerberus, a personal C++/Vulkan graphics engine and path tracer.

PROFESSIONAL EXPERIENCE

Virtuos — Jun 2022 – Present

Senior Graphics Programmer · *Third Kind Games* — *Feb 2025 – Present*

Unannounced UE5 project (under NDA) — *Jul 2026 – Present*

- Reduced Virtual Shadow Map cost for skinned foliage by **~29%** on PS5 by moving its invalidation to the static path and time-slicing GPU invalidation across frames when lighting changes.
- Profiled the Nanite Visibility pass, identifying voxels (**~28%**) and programmable raster (**~20%**) as its largest costs; built tools to switch foliage Nanite settings to voxels and tuned programmable raster, density and rasterizer configuration, improving the pass by **~20–25%**.
- Quantified programmable-raster cost in the Nanite visibility buffer with A/B tests, measuring **~10%** of the pass from masked materials and World Position Offset.
- Broke down Base Pass shading with A/B tests, measuring **~7–9%** savings each from derivative-heavy shading and vertex-side material features, and traced most of the cost to a small set of shading groups.
- Turned all content findings into a prioritized action plan for Art and Tech Art covering foliage voxelization, material consolidation and removal of unnecessary WPO and masking.
- Reduced Virtual Texture upload spikes by **~30–40%**.

Mavrix · *Unreal Engine 5, C++, HLSL, DirectX 12* — *Feb 2025 – Jul 2026*

- Worked extensively on memory within tight, fixed console budgets (notably Xbox Series S): fixed multiple memory leaks, improved memory management in Errant (custom plugin) and used memory profiling tools to drive game-wide optimizations.
- Diagnosed and fixed low-level rendering crashes and visual artifacts, including hardware-specific issues on PC, through engine-level changes.
- Backported several Unreal Engine fixes and improvements to resolve memory issues, improve performance and increase stability.
- Designed a modular graphics settings framework supporting NVIDIA DLSS, AMD FSR 4 and frame generation, with dynamic quality presets and runtime configuration.
- Built platform-specific graphics configurations and runtime Performance/Quality modes for PC (Windows GDK), PS5 and Xbox Series X|S using Device Profiles and scalability settings.
- Optimized Virtual Shadow Maps with a distance-based ray-count falloff for Shadow Map Ray Tracing and custom cache invalidation for large, dynamically streamed environments.
- Implemented selective motion blur exclusion by modifying the post-processing pipeline and HLSL shaders, using Custom Stencil to control which objects are blurred.
- Built runtime graphics debugging and stress-test tools and editor tools (icon generation, World Partition runtime-grid assignment), and worked daily with the art team to keep content within budget.

Senior Graphics Programmer · *Black Shamrock* — *Nov 2024 – Feb 2025*

Judas, Unreal Engine 4

- Made runtime reflection-probe generation time-sliced and on-demand, meeting the visual-quality needs of procedurally generated (PCG) content while improving performance.
- Mentored a junior developer in debugging and performance optimization.

Graphics Programmer · *Black Shamrock* *Aug 2023 – Nov 2024*

Judas, Unreal Engine 4

- Developed a render-state streaming system that reduced the primitives processed per frame and improved render-thread efficiency.
- Backported UE5 renderer features to UE4, including optimized compute visibility and refined render-state creation and destruction.
- Optimized reflection-capture generation without losing visual quality, and added features that improved art-team workflows.

Junior Graphics Programmer · *Virtuos Paris* *Jun 2022 – Aug 2023*

Judas, Unreal Engine 4

- Applied culling techniques that reduced draw calls.
- Analyzed and optimized lighting setups to stay within performance budgets, working with cross-functional teams.

Freelance Developer · *Compostela Films* *Jun 2019 – Jan 2022*

Unreleased mobile game, Unity

- Designed the architecture of the procedural level-generation algorithm.
- Built a C# audio system integrating Wwise that reduced runtime cache data and build size.

SELECTED PROJECTS

Cerberus — *personal Vulkan renderer and path tracer (C++)*

- Hardware ray-traced path tracer on Vulkan KHR ray tracing (BLAS/TLAS with compaction, shader binding table, bindless textures), alongside a forward PBR renderer with shadow cascades and IBL.
- A-SVGF denoiser on demodulated diffuse/specular signals: temporal reprojection with virtual specular reprojection, ReLAX-style history fix and variance-guided à-trous filtering.
- Render graph with declared resource access, automatic barrier derivation, Dynamic Rendering scope merging, pass culling and per-pass GPU timing, designed after Frostbite's FrameGraph and UE5's RDG.
- Thin Vulkan RHI with typed command lists (graphics, compute, transfer, ray tracing), handle-based resource ownership and deferred deletion, plus an ImGui editor.

Other graphics work

- Two game engines built from scratch in C++ with OpenGL and Vulkan, featuring particle systems, bloom and cel shading.
- Distributed offline CPU ray tracer in C++, and a progressive path tracer in Shadertoy.
- Master's thesis: Unity prototype that procedurally generates environmental storytelling through a multi-agent simulation.

SKILLS

Rendering (UE4–5)	Nanite, Virtual Shadow Maps, Lumen and ray tracing, Virtual Textures and Runtime Virtual Textures, culling, visibility and LOD, skinned meshes, foliage, upscaler and frame-generation integration (DLSS, FSR 4)
Languages	C++, C#, HLSL, GLSL
Graphics APIs	DirectX 12, Vulkan (incl. KHR ray tracing), OpenGL; graphics and compute shaders
Platforms	PC, PS5, Xbox Series X|S, last-gen consoles
Profiling	Unreal Insights, RenderDoc, PIX, Razor
Engines and tools	Unreal Engine 4 and 5 (engine source), Unity, Git, Perforce

EDUCATION

Master's in Computer Science, Instituto Superior Técnico, Lisbon *(2019 – 2021; Game Development, Interaction and Visualization)*

Bachelor's in Computer Science, University of Lisbon *(2016 – 2019)*